

LAPORAN SOFTWARE QUALITY ASSURANCE

METODE GREYBOX – REGRESSION TESTING



Anggota kelompok:

20231310046
20231310047

M. Irvan Alfiansyah
M. Nur Yanfa

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

1. PENDAHULUAN

1.1 Latar Belakang

Grey Box Testing adalah pendekatan pengujian yang menggabungkan karakteristik Black Box Testing dan White Box Testing. Penguji memiliki pengetahuan parsial atas struktur internal sistem — cukup untuk merancang test case yang lebih efektif, tetapi tetap mengevaluasi sistem dari perspektif perilaku eksternal yang terukur.

Dalam konteks proyek SayurSehat, Grey Box Regression Testing dilakukan setelah serangkaian perubahan pada kode, test, konfigurasi CI, dan dokumentasi. Tujuannya adalah memastikan tidak ada fitur yang sebelumnya bekerja dengan benar menjadi rusak (broken) akibat perubahan-perubahan tersebut — kondisi yang dikenal sebagai regression.

1.2 Tujuan Pengujian

1. Memverifikasi bahwa fitur Checkout ACID tetap berfungsi setelah seluruh perubahan kode dan konfigurasi.
2. Memastikan tidak ada regression pada skenario kritis: checkout sukses, cart kosong, stok kurang, Midtrans gagal.
3. Memvalidasi bahwa pipeline CI (GitHub Actions) berhasil menjalankan test dan build secara otomatis.
4. Mengonfirmasi frontend React tetap dapat di-build untuk production setelah perubahan konfigurasi API URL.
5. Menyediakan traceability antara regression test case dengan pengujian Black Box dan White Box sebelumnya.

1.3 Perbedaan Grey Box vs Black Box vs White Box

Aspek	Black Box	White Box	Grey Box (ini)
Pengetahuan Kode	Tidak ada	Penuh	Parsial (terbatas)
Perspektif Pengujian	Pengguna akhir	Pengembang	Keduanya
Sumber Test Case	Input/output spec	Struktur kode	Spec + internal logic
Contoh pada Laporan Ini	Performance, Endurance Test	Data Flow Test	Regression Testing
Kekuatan Utama	Realistik, user-centric	Deteksi defect internal	Menyeluruh — coverage luas

1.4 Posisi Regression Testing dalam Siklus Pengujian SayurSehat

Regression Testing adalah tahap terakhir dan paling integratif dalam siklus pengujian proyek ini. Laporan ini diposisikan sebagai penutup setelah semua pengujian lain selesai:

Urutan	Jenis Test	Metode	Peran dalam Regression
1	Black Box	Performance Test	Baseline respons endpoint checkout saat load tinggi
2	Black Box	Endurance Test	Baseline stabilitas checkout selama 40 menit
3	Black Box	Sample / BVA / Class Partitioning	Baseline perilaku input checkout
4	White Box	Basic Path Test	Baseline jalur logika dan cyclomatic complexity
5	White Box	Data Flow Test	Baseline aliran data cart-order-payment
6	White Box	Loop Testing	Baseline perilaku loop cart dan batch order
7	Grey Box	Regression Testing (ini)	Memverifikasi semua baseline di atas tidak rusak

2. DASAR TEORI

2.1 Regression Testing

Regression Testing adalah pengujian ulang terhadap fitur yang sudah pernah diuji untuk memastikan perubahan baru tidak merusak perilaku yang sudah berjalan dengan benar sebelumnya. Dalam konteks grey box, regression testing tidak hanya mengamati output dari luar seperti black box, tetapi juga menjalankan ulang test yang berhubungan dengan struktur internal kode.

Regression biasanya dipicu oleh: penambahan fitur baru, refactoring kode, perubahan dependency, pembaruan konfigurasi, atau perubahan environment deployment. Tanpa regression testing, setiap perubahan berisiko merusak fitur yang sebelumnya sudah stabil.

2.2 Risiko Regression pada Modul Checkout ACID

Checkout ACID adalah modul kritis dalam SayurSehat. Perubahan kecil di mana pun dalam stack dapat menyebabkan:

- Checkout sukses berubah menjadi gagal tanpa sebab yang jelas.
- Cart kosong tidak lagi ditolak — order kosong masuk ke database.
- Stok tidak dikurangi dengan benar — stok ghost terjadi.
- Order dibuat tanpa order item — data tidak lengkap.
- Payment token tidak tersimpan — user tidak dapat membayar.
- Error bisnis (400/404) berubah menjadi 500 Internal Server Error.
- Race condition muncul kembali karena SELECT FOR UPDATE hilang dari repository.

2.3 Alasan Pemilihan Regression Testing pada Proyek Ini

Proyek SayurSehat telah mengalami beberapa perubahan penting yang menjadi pemicu regression testing:

Perubahan	Risiko Regression yang Ditimbulkan
Penambahan test Checkout ACID (Black Box)	Test baru harus tetap lulus bersama test lama yang sudah ada
Penambahan White Box test (Data Flow + Loop)	Mock/interface baru tidak boleh merusak kompatibilitas package
Penambahan CI GitHub Actions	Build dan test harus reproducible di lingkungan cloud GitHub
Perubahan deployment production	Endpoint live harus tetap stabil setelah konfigurasi baru
Perbaikan Swagger docs di CI pipeline	Build backend tidak boleh gagal karena package docs baru
Perubahan frontend API URL	Frontend React tetap harus bisa dibuild dengan konfigurasi baru

2.4 Strategi Regression Testing (Grey Box)

Regression testing pada proyek ini menggunakan kombinasi empat lapisan:

No	Suite	Command	Tujuan
1	Backend Full Regression	<code>go test ./...</code>	Seluruh package Go tetap lulus
2	Checkout Focused Regression	<code>go test ... -run 'TestCheckout TestDataFlow TestLoopTesting' -v</code>	Skenario checkout ACID inti tidak regression

3	Frontend Build Regression	<i>npm run build</i>	Frontend React tetap bisa dibuild untuk production
4	CI Quality Gate	<i>Push ke branch main</i>	Test dan build berjalan otomatis di GitHub Actions

3. RUANG LINGKUP PENGUJIAN

ID	Area Regression	Dicakup	Keterangan
RG-S01	Checkout usecase (unit test)	Ya	Cart kosong, transaction error, Midtrans fallback, checkout sukses
RG-S02	Data flow checkout (white box test)	Ya	Cart item ke order dan payment token — DF-CHK-01 s/d DF-CHK-08
RG-S03	Loop checkout (loop test)	Ya	Cart 0 item, 1 item, 3 item, dan batch supplier order
RG-S04	Seluruh package backend Go	Ya	go test ./... mencakup http, middleware, repository, usecase
RG-S05	Frontend React build	Ya	npm run build — memastikan React tidak gagal production build
RG-S06	CI otomatis (GitHub Actions)	Ya	Workflow success pada commit 22ed766
RG-S07	K6 live endpoint (performance/endurance)	Tidak	Tidak dijalankan ulang untuk menghindari pengurangan stok production

4. TARGET SOURCE CODE REGRESSION

Berikut adalah file dan fungsi yang menjadi target regression test, beserta risiko yang ada jika terjadi perubahan tidak terkontrol:

No	File	Fungsi / Bagian	Risiko Regression
1	<i>order_handler.go</i>	<i>OrderHandler.Checkout</i>	Auth/session berubah, response HTTP salah format
2	<i>order_handler.go</i>	<i>classifyCheckoutError</i>	Error bisnis (4xx) salah dipetakan menjadi 500
3	<i>order_usecase.go</i>	<i>Checkout</i>	Cart kosong, transaction, dan Midtrans fallback berubah perilaku
4	<i>order_repository.go</i>	<i>CheckoutTransaction</i>	Atomicity, pengurangan stok, order item, delete cart terganggu
5	<i>order_whitebox_additional_test.go</i>	<i>TestDataFlow, TestLoopTesting</i>	Test tambahan tidak kompatibel dengan interface usecase/repo
6	<i>.github/workflows/ci.yml</i>	<i>CI test & build jobs</i>	Generate Swagger docs atau build gagal di cloud

5. MATRIKS REGRESSION TEST CASE

Tabel berikut mendokumentasikan 12 test case dalam regression suite, mencakup area, skenario, expected result, referensi bukti, dan status akhir.

ID	Area	Skenario Regression	Expected Result	Status
RG-CHK-01	Usecase	Cart repository error — FindByUserID gagal	Return error, proses checkout berhenti, tidak ada order dibuat	PASS
RG-CHK-02	Usecase	Cart kosong — 0 item di keranjang	Return error cart kosong, tidak masuk ke CheckoutTransaction	PASS
RG-CHK-03	Usecase	Transaction repository gagal (stok kurang)	Checkout gagal dengan pesan valid — bukan success palsu	PASS
RG-CHK-04	Usecase	Checkout sukses — cart valid, stok cukup	Order berstatus PENDING dibuat dan dikembalikan	PASS
RG-CHK-05	Usecase	Midtrans Snap Token gagal (snapErr != nil)	Order tetap dikembalikan, payment_token nil	PASS
RG-CHK-06	Usecase	Snap response nil (snapResp == nil)	Order tetap dikembalikan tanpa modifikasi token	PASS
RG-CHK-07	Data Flow	Cart item dan VariantID mengalir ke token	Order ID dan total mengalir ke Snap; token & URL tersimpan	PASS
RG-CHK-08	Loop	Cart dengan 0 / 1 / 3 item — loop behaviour	Loop memproses sesuai jumlah item, tidak ada item hilang	PASS
RG-CHK-09	Loop	Batch supplier order campuran valid/invalid	Hanya order valid diproses; invalid tidak memengaruhi yang lain	PASS
RG-CHK-10	Package	Seluruh package Go — go test ./...	Semua package yang memiliki test file berhasil lulus	PASS
RG-CHK-11	Frontend	Build React production — npm run build	Build sukses, dist/ dihasilkan, tanpa error kompilasi	PASS
RG-CHK-12	CI	GitHub Actions — push ke main	Workflow success — backend Go test & build + frontend build	PASS

6. EKSEKUSI REGRESSION TESTING

6.1 Backend Full Regression — go test ./...

Pengujian ini menjalankan seluruh test suite backend Go secara serentak untuk memastikan tidak ada package yang rusak akibat perubahan apapun.

Command

```
$ cd backend-go
$ go test ./...
```

Output

```
? github.com/nuryanfa/e-commerce-sqa/cmd/api [no test files]
? github.com/nuryanfa/e-commerce-sqa/cmd/loadtest [no test files]
? github.com/nuryanfa/e-commerce-sqa/cmd/reset [no test files]
? github.com/nuryanfa/e-commerce-sqa/cmd/seed [no test files]
? github.com/nuryanfa/e-commerce-sqa/config [no test files]
? github.com/nuryanfa/e-commerce-sqa/docs [no test files]
ok github.com/nuryanfa/e-commerce-sqa/internal/delivery/http
ok github.com/nuryanfa/e-commerce-sqa/internal/middleware
ok github.com/nuryanfa/e-commerce-sqa/internal/repository
ok github.com/nuryanfa/e-commerce-sqa/internal/usecase
? github.com/nuryanfa/e-commerce-sqa/pkg/jwt [no test files]
? github.com/nuryanfa/e-commerce-sqa/pkg/password [no test files]
? github.com/nuryanfa/e-commerce-sqa/pkg/response [no test files]
```

PASS

Package	Hasil	Status
<i>internal/delivery/http</i>	ok — handler checkout dan route regression lulus	PASS
<i>internal/middleware</i>	ok — auth middleware tidak mengalami regression	PASS
<i>internal/repository</i>	ok — repository checkout transaction tidak regression	PASS
<i>internal/usecase</i>	ok — seluruh checkout usecase test lulus	PASS

6.2 Checkout Focused Regression

Pengujian ini menjalankan hanya test case yang berkaitan langsung dengan modul Checkout ACID, Data Flow, dan Loop Testing — tiga pilar utama pengujian White Box SayurSehat.

Command

```
$ cd backend-go
$ go test ./internal/usecase -run "TestCheckout|TestDataFlow|TestLoopTesting" -v
```

Output

```
=== RUN    TestCheckout_CartRepoError
--- PASS: TestCheckout_CartRepoError (0.00s)
=== RUN    TestCheckout_EmptyCart
--- PASS: TestCheckout_EmptyCart (0.00s)
=== RUN    TestCheckout_TransactionFails_StokKurang
--- PASS: TestCheckout_TransactionFails_StokKurang (0.00s)
=== RUN    TestCheckout_Success
--- PASS: TestCheckout_Success (0.00s)
=== RUN    TestCheckout_MidtransFails_OrderTetapKembali
--- PASS: TestCheckout_MidtransFails_OrderTetapKembali (0.00s)
=== RUN    TestCheckout_SnapRespNil_OrderTetapKembali
--- PASS: TestCheckout_SnapRespNil_OrderTetapKembali (0.00s)
=== RUN    TestDataFlow_CheckoutCartToOrderAndPaymentToken
--- PASS: TestDataFlow_CheckoutCartToOrderAndPaymentToken (0.00s)
=== RUN    TestLoopTesting_CheckoutCartItemCounts
--- PASS: TestLoopTesting_CheckoutCartItemCounts (0.00s)
=== RUN    TestLoopTesting_BatchProcessSupplierOrders
--- PASS: TestLoopTesting_BatchProcessSupplierOrders (0.00s)
=== RUN    TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
--- PASS: TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder (0.00s)

PASS
ok      github.com/nuryanfa/e-commerce-sqa/internal/usecase
```

No	Nama Test Function	Area	Status
1	<i>TestCheckout_CartRepoError</i>	Usecase — Error Handling	PASS
2	<i>TestCheckout_EmptyCart</i>	Usecase — Validasi Cart	PASS
3	<i>TestCheckout_TransactionFails_StokKurang</i>	Usecase — Transaction Error	PASS
4	<i>TestCheckout_Success</i>	Usecase — Happy Path	PASS
5	<i>TestCheckout_MidtransFails_OrderTetapKembali</i>	Usecase — Midtrans Fallback	PASS
6	<i>TestCheckout_SnapRespNil_OrderTetapKembali</i>	Usecase — Snap Response Nil	PASS
7	<i>TestDataFlow_CheckoutCartToOrderAndPaymentToken</i>	White Box Data Flow	PASS
8	<i>TestLoopTesting_CheckoutCartItemCounts</i>	White Box Loop Testing	PASS
9	<i>TestLoopTesting_BatchProcessSupplierOrders</i>	White Box Loop Testing	PASS
10	<i>TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder</i>	White Box Loop Testing	PASS

6.3 Frontend Build Regression — npm run build

Pengujian ini memastikan aplikasi frontend React tidak mengalami regression akibat perubahan konfigurasi API URL atau dependency.

Command

```
$ cd frontend-react
$ npm run build
```

Output

```
vite v7.3.1 building client environment for production...
2870 modules transformed.
dist/index.html
dist/assets/index-BPJe8Tst.css
dist/assets/index-CpOcJdo6.js
built in 7.21s

WARNING: Some chunks are larger than 500 kB after minification.
(Bundle size warning — bukan error regression)
```

Artefak	Keterangan	Status
<i>dist/index.html</i>	Entry point HTML berhasil dihasilkan	PASS
<i>dist/assets/index-BPJe8Tst.css</i>	CSS production berhasil di-bundle	PASS
<i>dist/assets/index-CpOcJdo6.js</i>	JavaScript production berhasil di-bundle (2870 modules)	PASS
<i>Build duration</i>	7.21 detik — wajar untuk 2870 modules	PASS
<i>Bundle size warning</i>	Beberapa chunk > 500 kB setelah minifikasi — ini WARNING, bukan ERROR. Build tetap sukses dan tidak termasuk regression failure.	Minor

6.4 CI Quality Gate — GitHub Actions

GitHub Actions menjadi quality gate otomatis yang berjalan pada setiap push ke repository. Ini memastikan regression terdeteksi sedini mungkin, bahkan sebelum code review.

Commit	Status	Durasi	Job	Keterangan
22ed766	Success	1 menit 40 detik	Backend Go Test + Build Frontend React Build	Semua job passed — tidak ada failure

File CI yang digunakan: `.github/workflows/ci.yml`

7. ANALISIS HASIL REGRESSION

7.1 Backend Full Regression

Seluruh 4 package backend yang memiliki test file berhasil lulus. Package tanpa test file (cmd/*, config, docs, pkg/*) dikecualikan dengan notasi [no test files] yang merupakan perilaku normal dari Go test runner — bukan indikasi masalah.

Keberhasilan ini membuktikan bahwa: penambahan test checkout ACID, penambahan White Box test (Data Flow + Loop), perbaikan Swagger CI, dan perubahan konfigurasi deployment tidak menyebabkan breaking change pada package manapun.

7.2 Checkout Focused Regression

Seluruh 10 test function yang berhubungan dengan Checkout ACID berhasil lulus dengan durasi 0.00s per test — menandakan semua menggunakan mock tanpa I/O nyata, sehingga deterministik dan tidak bergantung pada state eksternal.

Cakupan 10 test ini mencakup semua jalur kritis: error handling, validasi input, happy path, dan dua jenis fallback Midtrans. Tidak ada satu pun yang mengalami regression.

7.3 Frontend Build Regression

Build React berhasil menghasilkan seluruh artefak production (HTML, CSS, JS). Warning bundle > 500 kB adalah peringatan ukuran yang tidak berdampak pada fungsionalitas aplikasi dan merupakan masalah optimasi, bukan regression functional.

7.4 CI Quality Gate

GitHub Actions berhasil menjalankan backend Go test & build serta frontend React build dalam satu workflow berdurasi 1 menit 40 detik. Ini membuktikan pipeline CI berfungsi sebagai quality gate yang andal — setiap push berikutnya akan melewati gate yang sama secara otomatis.

7.5 Ringkasan Kuantitatif Regression

Suite Regression	Total Test	Lulus	Status
Backend full — go test ./...	4 package	4 package	PASS
Checkout focused — usecase test	10 test function	10 test function	PASS
Frontend build — npm run build	1 build	1 build	PASS
CI GitHub Actions — workflow	1 workflow	1 workflow	PASS

8. HUBUNGAN DENGAN BLACK BOX DAN WHITE BOX TESTING

Regression Testing dalam laporan ini bertindak sebagai jaring pengaman (safety net) yang memverifikasi bahwa semua baseline yang telah dibangun oleh pengujian Black Box dan White Box sebelumnya tetap berlaku.

Sumber Bukti Pengujian	Peran dalam Regression Testing
Black Box — Sample Testing	Menjadi baseline perilaku endpoint checkout dari perspektif pengguna akhir (input/output)
Black Box — Performance Testing	Menjadi baseline kemampuan respons checkout saat beban 10 VU selama 4 menit
Black Box — Endurance Testing	Menjadi baseline stabilitas checkout selama 40 menit berkesinambungan
White Box — Basic Path Testing	Menjadi baseline jalur logika dan cyclomatic complexity modul checkout
White Box — Data Flow Testing	Menjadi baseline kebenaran aliran data cart → order → payment token
White Box — Loop Testing	Menjadi baseline perilaku loop cart item dan batch processing supplier order
CI GitHub Actions	Menjalankan regression gate otomatis pada setiap push — memperpanjang coverage regression ke depan

9. ANALISIS ACID REGRESSION

Setiap prinsip ACID dianalisis dari perspektif risiko regression — apakah ada indikasi bahwa perubahan kode merusak jaminan ACID yang sebelumnya telah diverifikasi:

Prinsip ACID	Risiko Regression	Bukti Tidak Terjadi Regression
Atomicity	Transaksi parsial muncul setelah perubahan kode repository	TestCheckout_Success dan TestCheckout_TransactionFails lulus — membuktikan transaksi masih atomic. go test ./... juga lulus untuk package repository.
Consistency	Cart, stok, dan order tidak konsisten setelah perubahan usecase	TestCheckout_TransactionFails_StokKurang PASS — validasi stok masih aktif. TestCheckout_Success PASS — order hanya dibuat jika semua data konsisten.
Isolation	Locking SELECT FOR UPDATE hilang setelah refactoring repository	Source code masih menggunakan clause.Locking Strength=UPDATE. Endurance test sebelumnya (40 menit, 3 VU) tidak menunjukkan race condition.
Durability	Order tidak tersimpan setelah checkout akibat perubahan konfigurasi DB	TestCheckout_Success PASS — order dikembalikan dengan benar. SavePaymentToken dipanggil dan diverifikasi dalam TestDataFlow.

10. TEMUAN

ID	Temuan	Dampak	Rekomendasi	Kategori
RG-F01	Backend full regression lulus — seluruh 4 package Go berhasil	Tidak ada breaking change pada package backend setelah perubahan kode dan CI	Pertahankan go test ./... sebagai mandatory step di CI pipeline	Positif
RG-F02	Checkout focused regression lulus — 10 test function PASS	Seluruh skenario kritis checkout tetap berperilaku sesuai spesifikasi	Jalankan suite ini sebelum setiap deployment ke production environment	Positif
RG-F03	Frontend build lulus dengan warning bundle size > 500 kB	Build sukses dan aplikasi dapat dideploy — tidak ada functional regression	Pertimbangkan code splitting (React.lazy + dynamic import) di sprint berikutnya untuk optimasi performa	Minor
RG-F04	CI GitHub Actions berhasil berjalan — commit 22ed766 success	Quality gate otomatis aktif — regression akan terdeteksi pada setiap push	Jadikan CI success sebagai requirement wajib sebelum merge pull request ke main	Positif
RG-F05	K6 live endpoint tidak dijalankan ulang dalam regression ini	Tidak ada risiko langsung — K6 tidak berjalan pada test suite ini	Siapkan product khusus staging/test dengan stok dummy agar K6 live bisa dimasukkan ke regression berkala	Disengaja

11. KESIMPULAN

11.1 Ringkasan Hasil

KEPUTUSAN PENGUJIAN: LULUS (PASS) — Grey Box Regression Testing pada modul Checkout ACID dinyatakan LULUS. Seluruh 12 regression test case berhasil dieksekusi tanpa satu pun kegagalan. Pengujian Grey Box dengan metode Regression Testing membuktikan bahwa:

- 6. Seluruh package backend Go tetap lulus go test ./... setelah seluruh perubahan kode, test baru, dan konfigurasi CI.
- 7. 10 test function Checkout ACID (termasuk Data Flow dan Loop Testing) tetap PASS — tidak ada regression pada skenario kritis checkout.
- 8. Frontend React berhasil dibuild untuk production — perubahan konfigurasi API URL tidak menyebabkan build failure.
- 9. CI GitHub Actions berjalan sukses pada commit 22ed766 — quality gate otomatis aktif dan andal.
- 10. Prinsip ACID (Atomicity, Consistency, Isolation, Durability) tetap terjaga berdasarkan bukti test dan source code yang ada.
- 11. Tidak ditemukan regression pada jalur utama Checkout ACID — fitur tetap stabil dan layak dijadikan baseline untuk pengujian berikutnya.

11.2 Perbandingan Lintas Seluruh Laporan Pengujian

Laporan	Jenis	Metode	Hasil	Defect	Status
Performance Test	Black Box	Performance	4/4 threshold PASS	Tidak ada	PASS
Endurance Test	Black Box	Endurance	4/4 threshold PASS	Tidak ada	PASS
Data Flow Test	White Box	Data Flow	8/8 TC PASS	Tidak ada	PASS
Regression Test	Grey Box	Regression	12/12 TC PASS	Tidak ada	PASS

11.3 Rekomendasi Tindak Lanjut

- 12. Jadikan go test ./... dan npm run build sebagai mandatory CI check — tidak boleh ada merge ke main tanpa keduanya lulus.
- 13. Tambahkan test case regression untuk endpoint lain (cart management, product stock update) setelah fitur tersebut stabil.
- 14. Siapkan staging environment dengan data test terpisah agar K6 live dapat dimasukkan ke regression suite berkala tanpa risiko merusak data production.
- 15. Implementasikan code splitting di frontend untuk mengatasi warning bundle size > 500 kB — optimasi performa tanpa mengubah fungsionalitas.
- 16. Pertimbangkan automated regression report yang di-generate dari CI output untuk audit trail yang lebih lengkap.